Code availability is not enough: A schema-based representation for comprehensible and extensible data analysis

H. Sherry Zhang
Department of Statistics and Data Sciences, University of Texas at Austin
and
Roger D. Peng
Department of Statistics and Data Sciences, University of Texas at Austin

June 23, 2026

Abstract

Current reproducibility practices have greatly improved our ability to replicate published results, but they do not necessarily make data analyses easier to understand. Audience seeking to comprehend an analysis need to piece together information scattered across manuscripts and code scripts to understand the sequence of analytical steps and the rationale behind them, with some details may not explicitly recorded in either source. Exploring alternative analytical choices is even more challenging, as it requires understanding and modifying code that may be unfamiliar and consider how it would impact upstream and downstream code. To address these challenges, we propose a schema that represents data analyses as a collectino of analytical decisions, together with their justification, inputs, and outputs. Coupled with large language models (LLMs), the schema can be inferred from natural language descriptions and translated into analysis scripts. The structured representation also enables exploration of alternative analytical choices, allowing multiple analytical workflows to be generated and compared. We demonstrate the proposed workflow using the many-analysts football study, where LLMs create the schema from the decision chocies in team reports, generate corresponding analysis scripts, and explore alternative random effect specifications. As an intermediate representation between the manuscript and code scripts, our schema makes analyses more comprehensible and extendable from a reader's perspective.

Keywords: Reproducibility, Large Language Models (LLMs), Data analysis

1 Introduction

The reproducibility movement has been a major development in modern scientific research, motivated in part by concerns regarding the replication of published findings and the increasing reliance on computational methods across disciplines. In statistics, advocates of reproducible research argue that scientific claims should be accompanied by the data and code necessary to regenerate reported results (Peng 2011). These practices have greatly improved the reproducibility of published results across scientific domains (Kohler et al. 2026, Xu & Yang 2026) and have been incorporated into journal policies through data and code availability requirements (see the *Journal of the American Statistical Association* editorial (Wrobel et al. 2024)).

An implicit assumption underlying current practices is that reproducibility implies understandability, given that the data and the code required to reproduce the results are available to readers. Specifically, understandable implies that an independent researcher should be able to, without much difficulty, track the decisions made by the authors, understand the rationale behind those decisions, and confortably modify the code to explore alternative choices. As data analysis grown in complexity in modern applied research, understanding a piece of analysis has become non-trivial. Decisions related to data preprocessing and exploratory data analysis are often reported only briefly in manuscripts and left as technical details for readers to infer from supplementary materials. Many decisions are not made in a linear fashion but emerge through an iterative process (Zhang & Peng 2025, Jun, Birchfield, De Moura, Heer & Just 2022, Jun, Seo, Heer & Just 2022), and it is not entirely clear how this iterative nature of data analysis can be communicated. Making changes to an existing script can often be difficult because analytical decisions are intertwined with implementation details, project-specific abstractions, and dependencies across different parts

of the workflow, requiring readers to consider how modifications propagate through upstream and downstream components. As a result, readers seeking to understand, modify, or extend an analysis must often spend a non-trivial amount of the time to piece together information scattered across manuscripts, code scripts, and supplementary materials to fully understand an analysis. Recent advances in LLMs have shown strong capabilities for code generation tasks including code completion and repair (Jiang et al. 2026, Shah et al. 2026), opening new opportunities for making data analysis easier to comprehensible, modifiable, and adaptable.

In this paper, we present a schema designed to describe data analysis in a structured way that captures the decisions and their justifications, as well as the inputs and outputs of each step. We also demonstrate how this schema, coupled with LLMs, can be used to generate code scripts from natural-language descriptions in manuscripts, and explore alternative decision choices. We apply our schema to the many-analyst football study, where LLMs create the schema from the decision choices reported in the final reports of each team, and generate and validate the corresponding code scripts against the original published results. We further demonstrate how to the schema supports specification and exploration of alternatives random effect specification for Team 5.

The paper is organized as follows: in Section 2, we review the relevant literature on reproducibility and communicaition of data analysis, as well as the application of LLMs in data science. In Section 3, we present our methodology, including the schema representation for describing data analysis and the workflow for generating and validating code from the schema by LLMs. In Section 4, we apply our methodology to the many-analyst football and demonstrate that we can reproduce the results reported in the paper, identify errors in the original reports, and highlight failure modes of current LLMs. Finally, in Section 5,

LLMs to translate those decisions into code.

3 Methodology

As traditional reproducibility practices can be insufficient for understanding complex modern data analyses, we have designed two complementary YAML schemas and the associated workflow to better support comprehension and extensibility in applied data analysis. Our workflow is designed to be 1) decision-centric: analyses are represented as a sequence of decisions with explicit rationale, and 2) reader-oriented exploration: readers can inspect, compare, and extend those decisions.

3.1 A schema for describing data analysis

We represent a data analysis, $\mathcal{S} = \{A_1, A_2, \cdots, A_n\}$, as a sequence of n analytical step A_i, applied to a dataset $\mathcal{D}$, where each step, A_i, is defined as a tuple:

$$A_i = \langle f_i, \pi_i, r_i, \mathcal{I}_i, \mathcal{O}_i, \omega_i \rangle. \tag{1}$$

The components of A_i can be partitioned into two parts: a required triplet (f_i, π_i, r_i) and an optional triplet $(\mathcal{I}_i, \mathcal{O}_i, \omega_i)$. The first component of the triplet, $f_i \in \mathcal{F}$ is the *objective* of a step. The second component, $\pi_i \in \Pi$, represents the specific *decision* made to achieve the objective f_i and the term r_i is the *justification* provided for choosing the decision π_i. Structuring this information into a schema involves defining the three components described above for each step. While the decision (what has been done) and the justification (why it is done) are generally clear, the objective requires identifying what the step is trying to achieve. In other words, it represents the broader goal that could potentially be fulfilled

through other alternative choices (formally introduced in Section 3.2). As an example, consider the description for calculating the Human Development Index (HDI) (United Nations Development Programme 2025):

1) Scale all four variables (life expectancy, expected years of schooling, mean years of schooling, and GNI per capita) to the [0, 1] range.
2) Combine the two education variables (expected years of schooling and mean years of schooling) into a single education variable.
3) Combine the three dimensions (health, education, and income) into a single index using a geometric mean.

In Step 3, the objective is to combine the three dimensions. While a simple arithmetic mean could be an alternative decision to fulfill this goal, the geometric mean (the decision) is chosen because it penalizes uneven development among the three dimensions (the justification). The full schema for constructing HDI is presented in Listing 1.

Listing 1 An example schema for calculating Human Development Index (HDI)

```
meta:
  type: schema
nodes:
- tag: step-scaling
  objective: how to handle variables in different scales?
  decision: apply min-max scaling to each variable
  rationale: to convert the variables to the same scale for future
    construction of a single index
- tag: step-education
  objective: how to combine the two education-related variables?
  decision: take the average
  rationale: the arithmetic mean allows perfect substitutability
    between the two variables
- tag: step-combine
  objective: how to combine the three dimensions into a single index?
  decision: use the geometric mean
  rationale: the geometric mean penalizes uneven development
    among the three core dimensions
```

On the optional components of a data analysis step A_i, the operational flow of the analysis is governed by the input and output spaces, $\mathcal{I}_i$ and $\mathcal{O}_i$ respectively, once a dataset is provided. While the output of a step can be a pseudo object that not currently exist in the dataset, the inputs for any given step must be within the existing data scope, comprising variables in the original dataset and all outputs from previous steps. Formally, we require:

$$\mathcal{I}_i \subseteq \left(\mathcal{D} \cup \bigcup_{j<i} \mathcal{O}_j \right), \quad \forall i \in \{2, \ldots, n\}.$$

When the schema is extracted automatically by Large Language Models (LLMs) from text, the inputs and outputs will be inferred by the LLMs, if a dataset is provided. We also introduce a categorical confidence score, ω_i (low, medium, high) that can be useful in case when the source text is ambiguous to guide the LLM to ask for clarification to consolidate the schema extraction.

3.2 Describing alternatives

At each step, one decision choice is made by the researchers to address an objective, however the choice is often not the only possible one. For step i, there could be k_i decision choices that fulfill the objective f_i: $\{A_i^1, A_i^2, \ldots, A_i^{k_i}\}$. We can define a selection vector $r = (r_1, r_2, \ldots, r_n)$, where $r_i \in \{1, \ldots, k_i\}$, to describe an analysis comprised of specific series of decisions: $\mathcal{S}^r = \{A_1^{r_1}, A_2^{r_2}, \ldots, A_n^{r_n}\}$. A multiverse, used to describe the entire decision space, is defined as the Cartesian product of these sets: $\mathcal{M} = \{\mathcal{S}^r \mid r_i \in 1, \ldots, k_i, \forall i\}$. The cardinality of this space, $|\mathcal{M}| = \prod_{i=1}^{n} k_i$, quantifies the total number of unique, fully-specified analysis.

Returning to the HDI example, in Step 3, instead of a geometric mean, an arithmetic mean is another possibility to combine the three dimensions into a single index. It may not be the best decision, but still a plausible one to derive a single index from the three dimensions, hence achieve the objective of this step. Listing 2 shows the structure of an alternative YAML file. The `meta` section identifies the file type (an alternative file rather than a schema file) and specifies the analysis step and the objective under consideration. Individual alternatives are defined as entries in the alternatives list, allowing multiple alternatives to

be specified within a single file.

Listing 2 An example YAML file for specifying alternative for Step 3 in calculating Human Development Index (HDI).

```
meta:
  type: alternative
  step: step-combine
  objective: how to combine the three dimensions into a single index?
alternatives:
  - id: arith-mean
    decision: use an arithmetic mean
    rationale: a simple average of the three dimensions
```

3.3 Workflow

The overall workflow for schema-based data analysis is illustrated in Figure 1. A schema can be created either manually from the template or automatically by an LLM from a natural-language manuscript. Once created, the schema serves as a structured specification that guides a second LLM in generating the code for the analysis. To improve reliability, our design includes a third LLM agent which is responsible for code validation. While the latest LLMs can generate reasonable code based on prompt, they do not consistently achieve perfect accuracy. The validation process runs the generated script in a sandbox environment and if an error occurs, a third LLM agent will be employed to diagnose the script. The validation agent receives the generated code together with the error message and the correponding line number produced during the execution, and returns a newer version of the script. The revised script is then re-executed in the sandbox and this cycle continues until one of the three conditions is met: the code runs successfully, a maximum number

of attempt is reached, or the user terminates the process. As will be show in Section 4, this validation process resolves the majority of R syntax issues encountered during data wrangling and statistical modelling tasks. On top of supporting a single analytical specification, the workflow also allows users to agument a schema with an alternative YAML file. This expands the original schema into a multiverse of parallel schemas, each representing a different sequence of decision choices. The same code generation and validation workflow is then applied independently to each schema, producing a corresponding set of validation scripts.

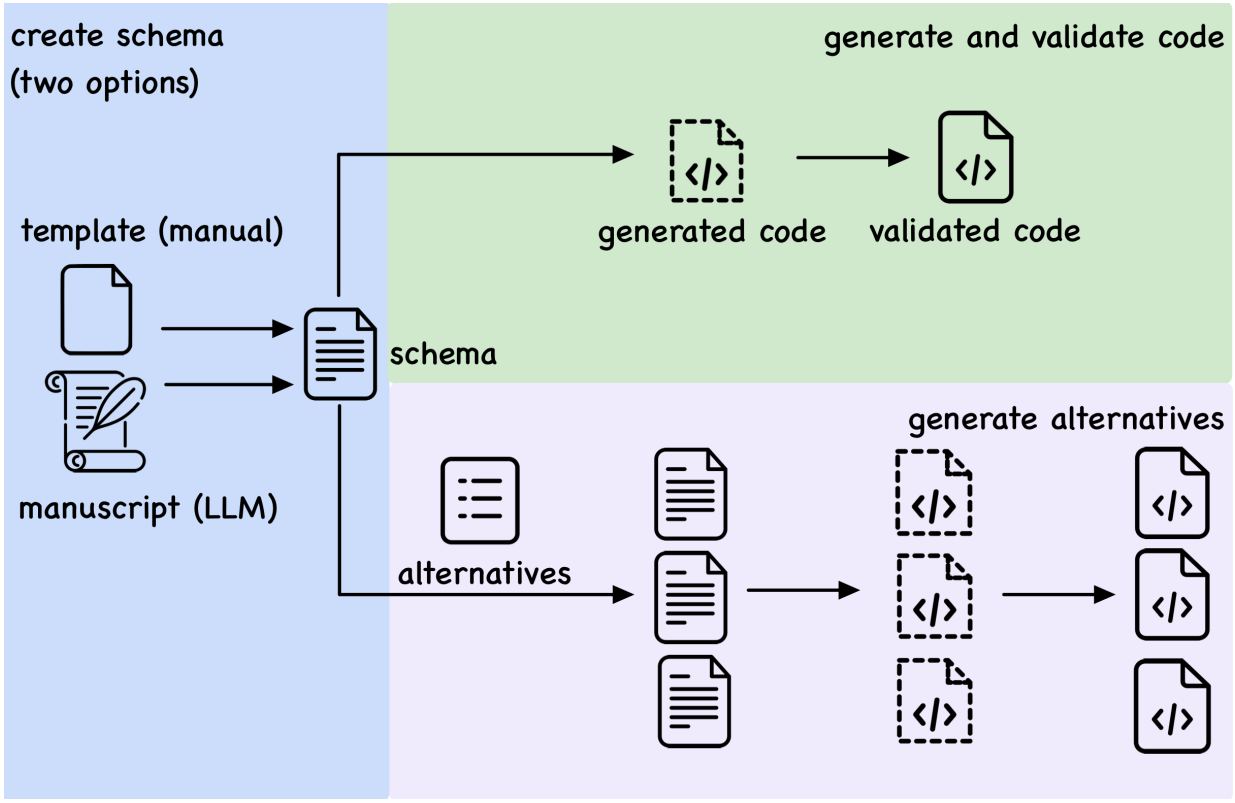


Figure 1: Overview of the schema-driven analysis workflow. Schemas can be authored manually or generated from natural language manuscripts by an LLM. Schemas are used to generate code, which is validated through iterative execution in a sandboxed environment and repaired when errors occur. Alternative specifications can expand a schema into a multiverse of parallel analyses.

Returning to the HDI example, the proposed workflow allows both authors and readers to interact with a data analysis through natural language. The schema in Listing 1 is straightforward to construct from a template and can be included as supplementary material to docuemnt the analytical workflow. An author may use the "code generation and validation" path in Figure 1 to automatically generate the analysis script for computing HDI and validate its accuracy. A reader, on the other hand, may wish to explore how the HDI values or country rankings change if the original arithmetic mean (instead of the geometry mean) is used. By combining the original schema in Listing 1 with the alternative specification Listing 2, the "alternative generation" path produces a corresponding analysis script for each alternative with LLMs. In contrast, modifying an existing code script by hand typically requires manually coordinating changes across upstream and downstream code. By representing analyses as a sequence of structured analytical decisions, our workflow allows LLMs to automate much of this process, making it easier to understand the sequence of steps taken in the analysis and to explore alternatives.